

27. L-атрибутные грамматики: вычисление при восходящем анализе. Маркеры. Доступность атрибутов в стеке.

1. L-атрибутные грамматики (Контекст проблемы)

Определение: Атрибутная грамматика называется **L-атрибутной**, если каждый *наследуемый* атрибут символа в правой части правила зависит только от атрибутов символов, стоящих *слева* от него, и наследуемых атрибутов родителя (левой части правила).

При **восходящем анализе** (LR-разбор) возникают две проблемы реализации L-атрибутных грамматик:

1. **Момент вычисления:** Действия выполняются только при свертке (в конце правила). Нельзя выполнить действие в середине правила, так как анализатор еще не знает, какое правило применяется.
2. **Отсутствие контекста:** Наследуемый атрибут часто зависит от родителя. Но при восходящем разборе родитель появляется в стеке только *после* свертки (когда правая часть уже удаляется).

2. Метод маркеров

Для решения проблем вводится техника **маркеров** — специальных нетерминалов, порождающих пустое слово.

Алгоритм вставки маркеров

Пусть дана схема трансляции на основе грамматики G . Мы строим грамматику G_M следующим образом:

1. Рассматриваем каждое правило $A \rightarrow X_1 \dots X_k$.
2. Если перед символом X_i в схеме стоит семантическое действие (вычисление его наследуемого атрибута), то:
 - Вставляем перед X_i уникальный маркер M .
 - Добавляем в грамматику правило $M \rightarrow \lambda$.

- Семантическое действие переносим в это правило (теперь оно вычисляет *синтезируемый* атрибут маркера M).

В результате $L(G_M) = L(G)$, но все наследуемые атрибуты исходной грамматики становятся синтезируемыми атрибутами маркеров.

3. Доступность атрибутов в стеке

Это ключевой механизм, объясняющий, как найти данные при свертке.

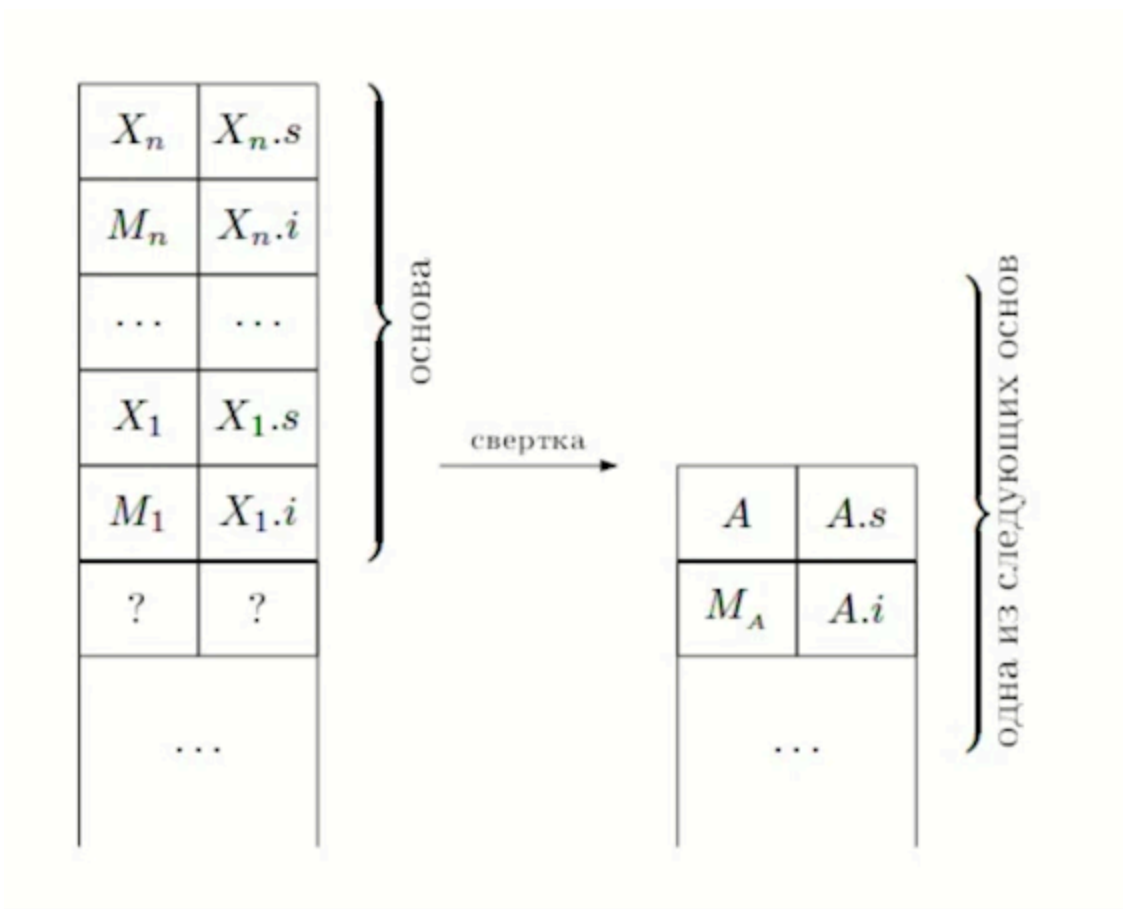
Организация стека

В стеке LR-анализатора пары $(state, val)$.

- val для терминалов/нетерминалов хранит их синтезируемые атрибуты.
- val для маркеров хранит значение, которое станет *наследуемым* атрибутом для следующего за ним символа.

Лемма о наследуемом атрибуте

Утверждение: Если у нетерминала A существует наследуемый атрибут, то он находится в стеке непосредственно под первым символом основы (handle).



Пояснение:

Пусть идет разбор по правилу с маркерами $A \rightarrow M_1 X_1 \dots M_n X_n$.

Когда анализатор готов выполнить свертку (вся правая часть в стеке), стек выглядит так (сверху вниз):

1. X_n (вершина)
2. M_n (хранит наследуемый атр. для X_n)
3. ...
4. X_1
5. M_1 (хранит наследуемый атр. для X_1)

Таким образом, любой атрибут $X_j.i$ доступен через поле `val` маркера M_j . Поскольку структура правила фиксирована, мы всегда знаем смещение относительно вершины стека (`top`).

Пример доступа:

Если верхний символ X_{j-1} , то его значение в `val[top]`. Значение маркера M_{j-1} (лежащего под ним) — в `val[top-1]`.

4. Пример работы (Трассировка)

Рассмотрим правило вычисления кегля шрифта: $S \rightarrow [A.ps = 10]A$.

Преобразование:

1. $S \rightarrow LA$
2. $L \rightarrow \lambda \quad \{L.s = 10\}$

Процесс разбора:

1. Анализатор видит начало строки, сворачивает пустоту в L (правило $L \rightarrow \lambda$).
2. В стек кладется L , его атрибут $L.s = 10$.
3. Далее идет разбор A .
4. Когда A будет свернут, в стеке окажется последовательность L, A .
5. При финальной свертке $S \rightarrow LA$, атрибут $A.ps$ берется из $L.s$ (который лежит глубже в стеке).

5. Свойства маркированных грамматик

Влияние введения маркеров на класс грамматики:

1. **LL(1):** Если исходная G была LL(1), то G_M **всегда остается LL(1)**. Конфликтов не возникает, так как у маркеров только одно правило вывода ($M \rightarrow \lambda$).
2. **LR(1):** Если G была LR(1), то G_M **не обязательно** будет LR(1).
 - Могут возникнуть конфликты разбора, так как мы "раздуваем" правую часть правил и меняем контекст.
 - Однако, класс "маркируемых" грамматик шире, чем LL(1). Большинство практических задач решается успешно.

Вывод: Метод маркеров позволяет универсально реализовать вычисление атрибутов в L-атрибутных грамматиках при восходящем анализе, сводя задачу к вычислению синтезируемых атрибутов и использованию стека для передачи контекста.

Исходники лекции:

Определение

Атрибутная грамматика называется *L-атрибутой*, если для любого правила вывода $A \rightarrow X_1 \dots X_n$ и любого $i = 1, \dots, n$ каждый наследуемый атрибут символа X_i из правой части зависит только от атрибутов символов $X_1, \dots, X_{i-1}$ и наследуемых атрибутов символа A . В частности, любая S-атрибутная грамматика является L-атрибутой.

Обход в глубину дерева вывода назовем *стандартным*, если сыновья любого узла просматриваются слева направо (т. е. если в данном узле применялось правило $A \rightarrow X_1 X_2 \dots X_n$, то его сыновья, помеченные $X_1, X_2, \dots, X_n$, просматриваются именно в этом порядке). Такой порядок просмотра является и порядком создания узлов дерева при нисходящем анализе.

Проблемы при вычислении атрибутов

Две основные проблемы, возникающие при попытке реализовать вычисление атрибутов в L-атрибутной грамматике при восходящем анализе.

1. Действия с атрибутами можно выполнять в конце правила вывода (при свертке), но невозможно «внутри» правила, как это делалось при нисходящем анализе. В момент появления символа в стеке может быть неизвестно, в правой части какого правила вывода он окажется, что приводит к невозможности вычисления наследуемых атрибутов до появления в стеке всей основы.
2. Наследуемый атрибут символа может зависеть от наследуемого атрибута родителя; родитель же данного символа появится в стеке только в момент удаления самого символа, т. е. вычисление такого наследуемого атрибута вообще не представляется возможным.

Замена L-атрибутной схемы на S-атрибутную

В некоторых случаях удастся снять все проблемы заменой L-атрибутной грамматики на S-атрибутную, как в данном примере:

	Правило вывода	Семантические правила
1	$D \rightarrow TL$	$L.i = T.type$
2	$T \rightarrow \text{int}$	$T.type = \text{int}$
3	$T \rightarrow \text{real}$	$T.type = \text{real}$
4	$L \rightarrow L_1; a$	$L_1.i = L.i$ $\text{ADDTYPE}(a.entry, L.i)$
5	$L \rightarrow a$	$\text{ADDTYPE}(a.entry, L.i)$

Эту грамматику можно перестроить, избавившись от наследования типа идентификаторов нетерминалом L .

Замена L-атрибутной схемы на S-атрибутную

Для этого включим наименование типа первым элементом в список идентификаторов. В итоге получим S-атрибутную грамматику:

	Правило вывода	Семантические правила
1	$D \rightarrow L a$	$addtype(a.entry, L.type)$
2	$L \rightarrow L_1 a;$	$L.type = L_1.type$ $addtype(a.entry, L.type)$
3	$L \rightarrow \text{int}$	$L.type = \text{int}$
4	$L \rightarrow \text{real}$	$L.type = \text{real}$

Маркеры: пример

В общем случае от наследуемых атрибутов избавиться нельзя, однако обе изложенные выше проблемы могут быть решены введением дополнительных нетерминалов-маркеров, помещаемых в правую часть правила – там, где в схеме трансляции предусмотрено вычисление наследуемых атрибутов. Вначале рассмотрим пример.

Рассмотрим схему трансляции, описывающую вычисление высоты текста в $\text{T}_\text{E}\text{X}$. Проанализируем строку из одного символа x восходящим методом. Перенесем x в стек и свернем к B , однако выполнить соответствующее семантическое действие $([B.\text{ht}=\text{sym.h}*B.\text{ps}])$ не сможем: значению атрибута $B.\text{ps}$ неоткуда взяться в стеке, который был пуст до переноса x . Тем не менее очевидно, что для первого символа кегль известен заранее и равен 10; таким образом, необходимая для выполнения семантического действия информация доступна.

Маркеры: пример

Добьемся того, чтобы эта информация была доступна в стеке. Наследование базового кегля происходит в правиле $S \rightarrow [A.\text{ps} = 10]A$. Вставим перед символом A *маркер* L и добавим аннулирующее правило $L \rightarrow \lambda$. Это эквивалентное преобразование грамматики. Теперь LR-анализатор при разборе любой строки обнаружит, что первой выполняется свертка по правилу $L \rightarrow \lambda$; с символом L можно положить в стек значение базового кегля. Тем самым это значение будет доступно при всех последующих свертках.

Аналогичные проблемы с наследованием кегля возникают и в других правилах грамматики; в каждом случае для разрешения проблемы вводится уникальный маркер, функции которого аналогичны функциям маркера L .

Маркеры: пример

Правило вывода	Семантические правила	Фрагмент кода
$S \rightarrow LA$	$A.ps = L.s, S.ht = A.ht$	$val[ntop] = val[top]$
$L \rightarrow \lambda$	$L.s = 10$	$val[ntop] = 10$
$A \rightarrow A_1MB$	$A_1.ps = A.ps$ $M.i = A.ps, B.ps = M.s$ $A.ht = \text{MAX}(A_1.ht, B.ht)$	$val[ntop] =$ $\text{MAX}(val[top-2], val[top])$
$M \rightarrow \lambda$	$M.s = M.i$	$val[ntop] = val[top-1]$
$A \rightarrow B$	$B.ps = A.ps, A.ht = B.ht$	
$B \rightarrow B_1_NB_2$	$B_1.ps = B.ps$ $N.i = B.ps, B_2.ps = N.s$ $B.ht = \text{DISPOSE}(B_1.ht, B_2.ht)$	$val[ntop] =$ $\text{DISPOSE}(val[top-3], val[top])$
$N \rightarrow \lambda$	$N.s = \text{SHRINK}(N.i)$	$val[ntop] = \text{SHRINK}(val[top-2])$
$B \rightarrow \{PA\}$	$P.i = B.ps, A.ps = P.s$ $B.ht = A.ht$	$val[ntop] = val[top-1]$
$P \rightarrow \lambda$	$P.s = P.i$	$val[ntop] = val[top-1]$
$B \rightarrow \text{sym}$	$B.ht = \text{sym}.h * B.ps$	$val[ntop] = val[top] * val[top-1]$

Маркеры: пример

При каждой свертке требуется только одно «явное» вычисление (цепное правило $A \rightarrow B$ не требует вычислений). Часть вычислений, связанных с наследуемыми атрибутами, реализуется неявно. Происходит это благодаря тому, что

- (•) после любого шага работы анализатора в стеке под каждым «основным» нетерминалом находится маркер, значение атрибута которого есть значение наследуемого атрибута данного нетерминала.

В самом деле, если нетерминал A или B является не первым символом в основе, то прямо под ним в стеке лежит маркер, хранящий значение наследуемого атрибута этого нетерминала. Если же A или B – первый символ в основе, то его наследуемый атрибут копирует значение наследуемого атрибута родителя (т. е. может храниться там же).

Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие (•) выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$.

Свёртка по правилу
 $L \rightarrow \lambda$

state	val	Правило
L	10	$L.s = 10$

Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие (•) выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$.

Перенос x с добавлением
 $x.h$

state	val	Правило
x	0.6	
L	10	$L.s = 10$

Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие (•) выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$.

Свёртка по правилу
 $B \rightarrow x$

state	val	Правило
B	6	$B.ht = sym.h * B.ps$
L	10	$L.s = 10$

Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие (•) выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$.

Свёртка по правилу
 $N \rightarrow \lambda$

state	val	Правило
N	4	$N.s = SHRINK(N.i)$
B	6	$B.ht = sym.h * B.ps$
L	10	$L.s = 10$

Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие (•) выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$.

Перенос x с добавлением
 $x.h$



<i>state</i>	<i>val</i>	Правило
x	0.6	
N	4	$N.s = \text{SHRINK}(N.i)$
$_$		
$\overline{B}$	6	$B.ht = \text{sym}.h * B.ps$
L	10	$L.s = 10$

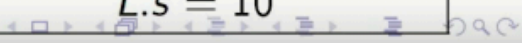
Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие $(\bullet)$ выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$. 

Свёртка по правилу
 $B \rightarrow x$

<i>state</i>	<i>val</i>	Правило
B	2.4	$B.ht = sym.h * B.ps$
N	4	$N.s = SHRINK(N.i)$
$_$		
$\bar{B}$	6	$B.ht = sym.h * B.ps$
L	10	$L.s = 10$



Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие (•) выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$.

Свёртка по правилу
 $B \rightarrow B_NB$

state	val	Правило
B	7.3	$B.ht = \text{DISPOSE}(B_1.ht, B_2.ht)$
L	10	$L.s = 10$

Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие (•) выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$.

Свёртка по правилу
 $A \rightarrow B$

state	val	Правило
A	7.3	
L	10	$L.s = 10$

Маркеры: пример

Так как родитель – это снова символ A или B , то для него справедливы те же рассуждения. Поскольку нижним символом в стеке является маркер L , рано или поздно рассматриваемый нетерминал будет свернут вместе с символом, лежащим под ним, который обязан являться маркером. Тем самым условие $(\bullet)$ выполняется во всех случаях.

Произведём разбор строки x_x в стеке. Вывод: $S \Rightarrow LA \Rightarrow LB \Rightarrow LB_NB \Rightarrow LB_Nx \Rightarrow LB_x \Rightarrow Lx_x \Rightarrow x_x$.

Свёртка по правилу
 $S \rightarrow LA$

state	val	Правило
S	7.3	

Обобщение примера

Использование маркеров позволяет, во-первых, производить семантические действия только в конце правил вывода, а во-вторых, хранить наследуемые атрибуты «основных» нетерминалов в стеке в форме атрибутов маркеров; при этом значение наследуемого атрибута символа хранится в стеке непосредственно под этим символом (даже если самого символа в стеке еще нет, т. е. к нему не произведена свертка, его наследуемый атрибут уже присутствует в стеке непосредственно под нижним символом основы).

Обобщим рассмотренный пример до универсального метода. Примем дополнительное предположение о рассматриваемой схеме трансляции:

- в любом правиле вывода для любого $i = 1, \dots, n$ действие с номером i присутствует тогда и только тогда, когда символ X_i имеет наследуемый атрибут.

Алгоритм вставки маркеров в грамматику

Вход: схема трансляции, основанная на грамматике G .

Выход: грамматика с маркерами G_M такая, что $L(G_M) = L(G)$.

- 1 для каждого правила вывода $A \rightarrow X_1 \dots X_k$ из G
- 2 цикл по i от 1 до k
- 3 если в схеме трансляции $[\text{действие}]_i \neq \emptyset$
- 4 вставить перед X_i уникальный маркер M_i
- 5 добавить правило $M_i \rightarrow \lambda$ в G_M

Очевидно, что языки, задаваемые грамматиками G и G_M , совпадают.

Замечания о маркированной грамматике

- Если G – $LL(1)$ -грамматика, то G_M также является $LL(1)$ -грамматикой.

В самом деле, у маркеров только по одному правилу вывода, поэтому конфликт множеств **SELECT** невозможен. Для всех остальных правил множества **FIRST** и **FOLLOW** никак не изменились, поскольку из маркеров выводится только пустое слово.

- Т.к. мы будем анализировать грамматику с маркерами восходящим методом, то достаточно, чтобы она была $LR(1)$ -грамматикой. Класс «маркируемых» грамматик шире класса $LL(1)$, например, в него входит грамматика из рассмотренного примера. Однако существуют $LR(1)$ -грамматики, в результате маркировки которых получаются не $LR(1)$ -грамматики.

Действия LR-анализатора для G_M

- Аксиома грамматики в восходящем анализе не появляется в правых частях правил вывода. Т.е., значения ее наследуемых атрибутов не меняются в ходе трансляции, поэтому их можно заменить на константы и считать, что у *аксиомы нет наследуемых атрибутов*.

Пусть для G_M построен LR-анализатор. Покажем, что в процессе LR-анализа можно вычислить в стеке все атрибуты. Шаг LR-анализатора грамматики G_M – это перенос, свертка к маркеру или свертка по правилу вида $A \rightarrow \alpha_M$, полученному из правила $A \rightarrow \alpha$ грамматики G .

Отметим, что таблица LR-анализа для G_M не содержит конфликтов, т. е. анализатор на шаге свертки к маркеру однозначно определяет, какой именно маркер нужно добавлять в стек. Кроме того, если A имеет наследуемый атрибут, то в стеке непосредственно под A обязан находиться маркер.

Действия в стеке LR-анализатора

Как и в анализе S-атрибутных грамматик, будем размещать в стеке рядом с символом (поле *state*) значение его синтезируемого атрибута (поле *val*). Значение наследуемого атрибута нетерминала будем размещать в поле *val* предыдущей записи стека (которая содержит маркер) и *вычислять в момент добавления маркера в стек*.

Пусть в схеме трансляции правило $A \rightarrow \alpha$ вместе с соответствующими семантическими действиями имеет вид $A \rightarrow [\text{действие}_1]X_1 \dots [\text{действие}_n]X_n[\text{действие}_{n+1}]$, и, следовательно, $\alpha_M = M_1X_1 \dots M_nX_n$. Покажем, как все действия могут быть выполнены в стеке LR-анализатора при добавлении маркеров $M_1, \dots, M_n$ в стек и свертке по правилу $A \rightarrow \alpha_M$.

Действия в стеке LR-анализатора

$A \rightarrow [\text{действие}_1]X_1 \dots [\text{действие}_n]X_n[\text{действие}_{n+1}],$

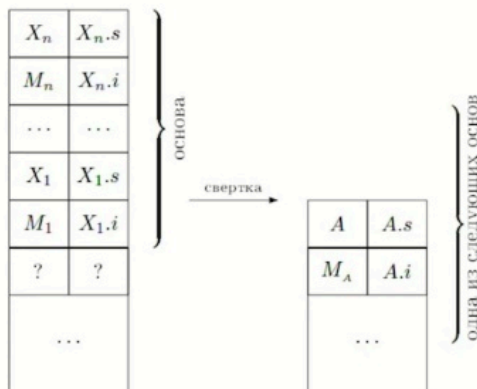
Действия с 1-го по n -е выполняются при добавлении маркеров. В j -м действии вычисляется наследуемый атрибут $X_j.i$, сопоставленный добавляемому в поле $state[val + 1]$ маркеру M_j . Этот атрибут может зависеть от значений атрибута $A.i$ и атрибутов $X_1.i, X_1.s, X_2.i, \dots, X_{j-1}.s$. Расположение атрибутов «иксов» в стеке определить легко.

Поскольку на рассматриваемом шаге анализа верхним символом в стеке является X_{j-1} , то значение $X_{j-1}.s$ размещается в $val[val]$; предыдущая запись в стеке содержит M_{j-1} и значение $X_{j-1}.i$; и т. д. (если какие-то из маркеров отсутствуют, мы все равно можем определить положение значений всех атрибутов).

Лемма о наследуемом атрибуте

Лемма

Если наследуемый атрибут нетерминала A существует, то он находится в стеке непосредственно под первым символом основы.



Символ A не является аксиомой; следовательно, после свертки по правилу $A \rightarrow M_1 X_1 \dots M_n X_n$ он окажется в основе для одной из последующих сверток. В силу наличия у A наследуемого атрибута, перед A в этой основе находится маркер, а с ним в стеке – атрибут $A.i$.

Заключительные замечания

С учетом леммы мы можем найти в стеке всю информацию, необходимую для вычисления $X_j.i$. Отметим, что в j -м действии могут, кроме вычисления $X_j.i$, содержаться какие-то фрагменты кода, но эти фрагменты могут обращаться только к $X_j.i$ и атрибутам, использовавшимся при вычислении $X_j.i$ согласно правилам размещения действий в схеме трансляции. Последнее, $(n+1)$ -ое действие может быть выполнено при свертке по правилу $A \rightarrow \alpha_M$, поскольку вся основа к этому моменту перенесена в стек, т. е. позиции всех необходимых для вычисления $A.s$ атрибутов в стеке известны. В итоге полностью реализована схема трансляции в процессе LR-анализа маркированной грамматики G_M .